Software Architecture and Design Specification

Project: Event Management System (EMS)

Authors: Prajin Ravi Shankar, Rakshan R, Palash Agarwal, Veladi Yathindra Deveswar

Date: 03-10-2026

Status: Final demo

Revision History

Version	Date	Author	Change Summary
1.0	05-10-2026	Prajin Ravi Shankar	Initial SAD draft based on the SRS and the SAD template.
1.1	06-10-2026	Rakshan R	Added the component diagram, component descriptions, architecture decisions (ADRs) and technology stack.
1.2	06-10-2026	Palash Agarwal	Added the security architecture (STRIDE), risks, traceability and the UML sequence diagrams (SD-1 to SD-3).
1.3	07-10-2026	Veladi Yathindra Deveswar	Added the API design, error handling, logging and monitoring, UX design, open issues and appendices.

Approvals

Approvals

Role	Name Signature/Date
Project Representative	
Instructor/Guide	

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Event Management System (EMS) so that the four team members can implement it consistently and so that reviewers can trace every requirement in the SRS to a component, design element and test.

1.2 Scope

Covers all EMS services: accounts and authentication, event creation and approval, event discovery, registration with waitlist, notifications, attendance check-in, reports and administration (features F1 to F8 of the SRS).

1.3 Audience

Developers, testers, security reviewers, course instructors and future maintainers.

1.4 Definitions

EMS, SRS, STP, RBAC, JWT, REST, STRIDE, ADR (Architecture Decision Record), DAL (Data Access Layer), TLS. See SRS section 1.4 for the full list.

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture (components, pattern, technology, security). Section 4 describes the detailed design (sequence diagrams, API, error handling, UX). Section 5 holds the glossary, references and tools.

2.2 Related Documents

EMS Software Requirements Specification v1.0 (requirement IDs EMS-F, EMS-NF, EMS-SAF, EMS-SEC); EMS Software Test Plan v1.0 (test case IDs TC-xxx); Requirement Traceability Matrix (SRS Appendix C).

3. Architecture

3.1 Goals & Constraints

Goals: correctness (never over-book an event), security (OWASP-aligned), responsiveness (search under 2 s, registration under 3 s at 100 concurrent users), maintainability (clear modules, 70% unit-test coverage on services), and deliverability by four students within one semester.

Constraints: MERN stack (problem statement #7); free tools only; one deployable server application; HTTPS; email via external SMTP; the code is built by 4 people in parallel, so modules must have clear boundaries.

3.2 Stakeholders & Concerns

Stakeholder	Concerns	Addressed by
Attendees	Easy registration, correct status, timely emails, privacy	C1 usability, C5, C6, SEC-001/002
Organizers	Quick event setup, accurate counts, attendance data	C4, C7, atomic seat update (ADR-4)
Admin	Control over events and users, audit trail	C3, C4, C10
Developers	Modularity, parallel work, testability	Layered modular monolith (ADR-1)
Instructors / security reviewers	Traceability, secure design, quality of documentation	Section 3.8, 3.9, SRS Appendix C

3.3 Component (UML) Diagram

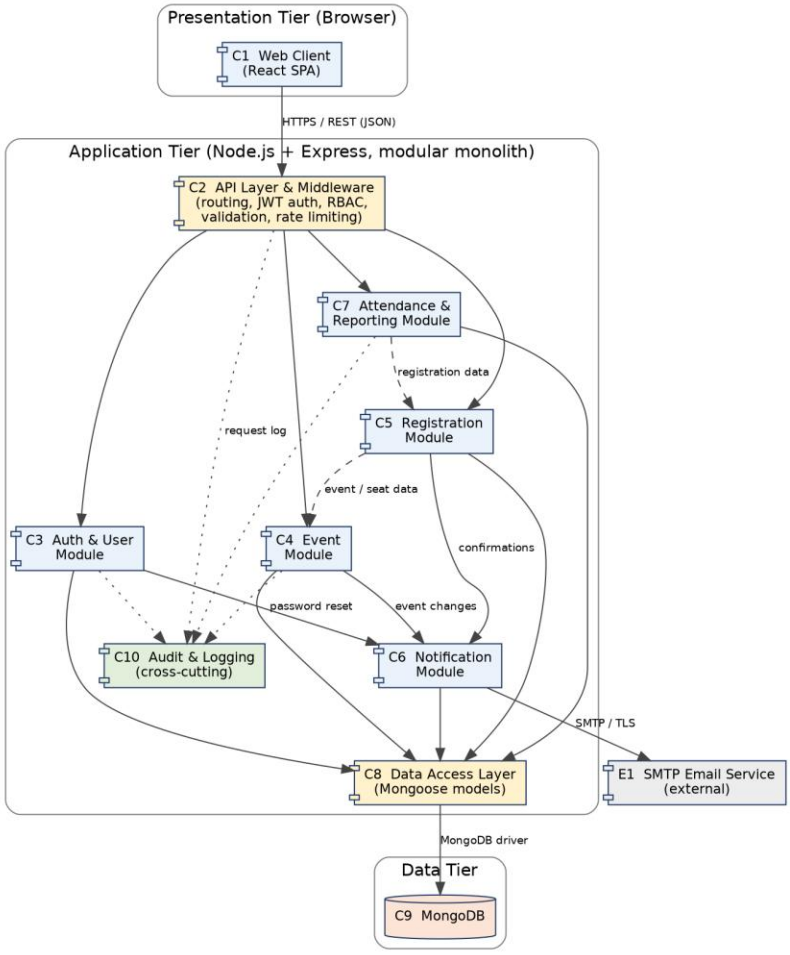


Figure 1: UML component diagram of EMS

3.4 Component Descriptions

ID	Component	Responsibility
C1	Web Client (React SPA)	Renders pages for all roles, validates input for user convenience, calls the REST API over HTTPS, stores no secrets.
C2	API Layer & Middleware	Express routers and middleware: request parsing, JWT authentication, RBAC and ownership checks, input validation and sanitisation, rate limiting, central error handler.
C3	Auth & User Module	Registration, login, lockout, password reset, profile, user administration (activate, deactivate, role change).
C4	Event Module	Event CRUD, validation of dates and capacity, approval workflow, cancellation, categories, public listing and search.
C5	Registration Module	Registration, atomic seat allocation, duplicate prevention, waitlist and FIFO promotion, cancellation, QR payload generation.
C6	Notification Module	Email templates, asynchronous queue, retry policy, reminder scheduler, delivery log. Talks to the SMTP service.
C7	Attendance & Reporting Module	Check-in by registration ID or QR, organizer dashboard, CSV export, admin platform summary.
C8	Data Access Layer	Mongoose schemas, indexes and repositories; the only component that talks to the database.
C9	MongoDB	Persistent store for users, events, categories, registrations, notification logs and audit logs.
C10	Audit & Logging	Structured application logs and the audit trail of security-relevant actions (cross-cutting).
E1	SMTP Email Service (external)	Third-party mail relay used to deliver emails (for example Gmail SMTP or Mailtrap in test).

Dependency rules: C1 talks only to C2. C2 calls the feature modules (C3 to C7). Feature modules access data only through C8. Only C6 talks to E1. Modules call each other only through the service interfaces listed in the diagram (for example C5 asks C4 for event and seat data; C4, C5 and C3 ask C6 to send email).

3.5 Chosen Architecture Pattern and Rationale

Pattern: three-tier client-server architecture with a layered, modular-monolith back end (Routes/Controllers, Services, Data Access). The web client is a single-page application that consumes a REST API.

ADR	Decision	Alternatives considered	Rationale
ADR-1	Layered modular monolith (one Node.js app, one module per feature)	Microservices	Four students and one semester: a single deployable unit is faster to build, test and run; module boundaries keep the option to split later.

ADR	Decision	Alternatives considered	Rationale
ADR-2	MongoDB with Mongoose	MySQL / PostgreSQL	Flexible schema while requirements evolve; JSON documents match the API; MongoDB transactions and atomic conditional updates provide the consistency required for registration and seat counting (ADR-4).
ADR-3	JWT-based authentication with user-status validation on protected requests, using an HttpOnly cookie with 60-minute expiry	Server-side sessions; tokens in localStorage	JWT-based authentication supports scaling (EMS-NF-008); HttpOnly cookie reduces token theft through XSS (EMS-SEC-007), while user-status validation ensures deactivated users are rejected.
ADR-4	Seat allocation and registration creation using a MongoDB transaction with an atomic conditional seat update.	Read-then-write in code; transactions	Prevents race conditions and guarantees EMS-F-021 by atomically updating the event seat count and creating the registration.
ADR-5	Asynchronous notification queue (database-backed job collection plus scheduler) with retry	Send email inside the request	Registration stays fast (EMS-NF-002); failures are retried and logged (EMS-F-025); no extra infrastructure such as Redis.
ADR-6	Route validation and a shared error handler.	Validation inside each controller	One place to apply EMS-SEC-004 and uniform error format (section 4.4).

3.6 Technology Stack & Data Stores

Layer	Technology
Client	React 19, React Router, Axios and CSS; Vite build.
Server	Node.js, Express 5, express-rate-limit, Helmet and cookie-parser.
Security libraries	bcryptjs with cost 12 for new and reset passwords; jsonwebtoken for JWTs.
Database	MongoDB replica set or Atlas with Mongoose 9. Unique email and registration code indexes, event status/start and category indexes, and transactional seat allocation. Search uses case-insensitive substring matching on title and description.
Email	Nodemailer over SMTP (Mailtrap in test, Gmail SMTP or equivalent in demo)
Scheduler	node-cron for 24-hour reminders and retry processing
Testing	Jest, Supertest (API), React Testing Library, JMeter (load), Lighthouse (accessibility), gitleaks / npm audit (security)
DevOps	GitHub (code, issues, projects, pull requests), GitHub Actions CI (lint, test, audit), .env files for secrets

All timestamps are stored in UTC and converted to IST in the client (EMS-NF-010).

3.7 Risks & Mitigations

Risk	Mitigation
MongoDB transaction with atomic conditional seat update (ADR-4); concurrency test TC-REG-06	Atomic conditional update (ADR-4); concurrency test TC-REG-06
SMTP service down or rate limited	Asynchronous queue with 3 retries and delivery log; core flows do not depend on email success
Slow search with many events	Indexes; pagination (10 per page); load test TC-PERF-01
Data loss	Daily automated backup and restore drill (EMS-NF-006)
Integration problems between four parallel developers	API contract in section 4.3 agreed first; feature branches and pull-request reviews; CI on every pull request
Secrets leaked to GitHub	.env in .gitignore, secret scan in CI (EMS-SEC-008)

3.8 Traceability to Requirements

Feature	Components	Requirements
F1 Accounts & Authentication	C2, C3, C6, C10	EMS-F-001 to 006, EMS-SEC-001, 003, 005, 007
F2 Event Creation & Approval	C2, C4, C6, C10	EMS-F-007 to 012, EMS-SEC-004
F3 Event Discovery	C2, C4, C8	EMS-F-013 to 015, EMS-NF-001
F4 Registration & Waitlist	C2, C5, C6, C8	EMS-F-016 to 021, EMS-NF-002
F5 Notifications	C6, E1, C8	EMS-F-022 to 025
F6 Attendance Check-in	C2, C7, C10	EMS-F-026, 027
F7 Reports & Dashboards	C2, C7	EMS-F-028 to 030
F8 Administration	C2, C3, C4, C10	EMS-F-031, 032, EMS-SEC-006
Cross-cutting	C1, C2, C8, C9, C10	EMS-NF-003 to 010, EMS-SAF-001, 002, EMS-SEC-002, 008

The complete requirement-to-component-to-design-to-code-to-test mapping is in SRS Appendix C and STP section 13.

3.9 Security Architecture

Trust boundaries: (1) Browser to server (untrusted client, HTTPS); (2) Server to database (database reachable only from the application host); (3) Server to SMTP service (external).

Threat model (STRIDE)

Threat	Example in EMS	Control	Req.
Spoofing	Guessing or stealing a user's password or token	bcrypt hashes, lockout after 5 failures, signed JWT with 60 min expiry in HttpOnly SameSite cookie, rate limiting on login	SEC-001, 005, 007, F-003
Tampering	Attendee edits another user's event or alters a request body	Server-side RBAC and ownership checks; schema validation; NoSQL operator stripping; HTTPS	SEC-002, 003, 004

Threat	Example in EMS	Control	Req.
Repudiation	Admin denies approving an event	Audit log of security-relevant actions (user, action, IP, time)	SEC-006
Information disclosure	Leaking password hashes or other users' data through API or logs	Response DTOs exclude sensitive fields; no secrets in logs; TLS; minimal personal data	SEC-001, 002, SAF-002
Denial of service	Request flooding, brute force	Rate limiting (100 requests per minute, 10 logins per minute per IP); pagination; request body size limit	SEC-005
Elevation of privilege	Attendee calls Organizer or Admin endpoints	RBAC middleware on every route; role changes only by Admin and audited	F-004, SEC-003

Security controls by layer

- **Client (C1):** output encoding by React; no secrets or tokens in localStorage; confirmation dialogs for destructive actions.
- **API layer (C2):** helmet security headers, CORS restricted to the client origin, rate limiter, JWT verification, RBAC and ownership guards, input validation, request size limit.
- **Modules (C3 to C7):** business-rule checks (for example capacity, deadline) done on the server; all queries go through Mongoose with sanitised input.
- **Data (C8, C9):** password hashes only; unique indexes; database user with least privilege; database port closed to the internet; backups.
- **Operations:** secrets in environment variables; dependency scan and secret scan in CI; logs without personal data beyond user ID.

4. Design

4.1 Design Overview

Each feature module follows the same three-layer structure: a controller (HTTP handling), a service (business rules) and a repository (data access through C8). Controllers contain no business rules, and services never use Express objects, which keeps services unit-testable (EMS-NF-007). The data model is shown below.

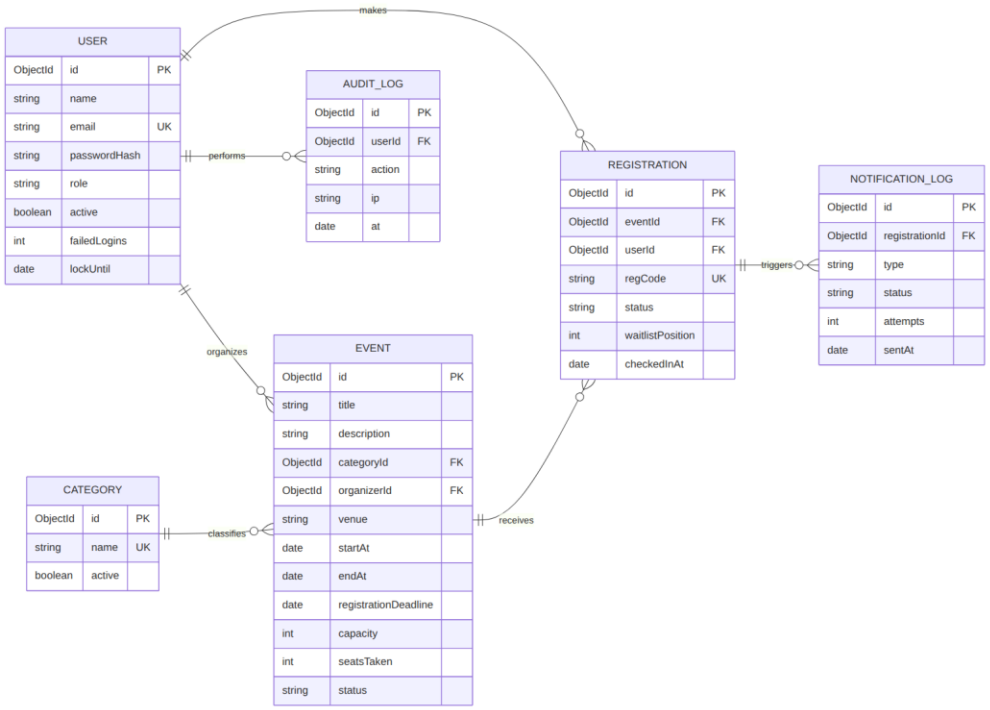


Figure 2: Data model (collections and relationships)

State models

Entity	States and allowed transitions
Event	Pending Approval -> Published (Admin approves) | Pending Approval -> Rejected (Admin rejects) | Published -> Cancelled (Organizer cancels). Cancelled and Rejected are final (BR-06).
Registration	Confirmed (seat free) or Waitlisted (event full) on creation; Waitlisted -> Confirmed (promotion); Confirmed or Waitlisted -> Cancelled; Confirmed -> Checked-in (during event window).

4.2 UML Sequence Diagrams

Three flows central to the project are shown: registration with waitlist (SD-1), event creation and approval (SD-2), and cancellation with waitlist promotion (SD-3).

SD-1: Register for an event (EMS-F-016, 017, 018, 021, 022)

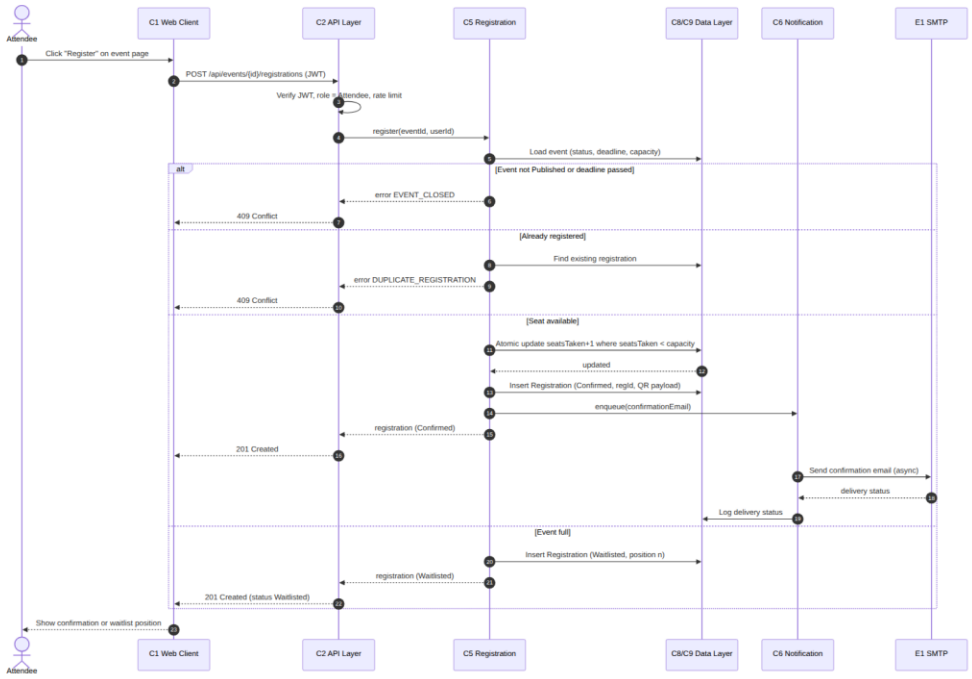


Figure 3: Sequence diagram SD-1 - Attendee registers for an event

SD-2: Create and approve an event (EMS-F-007 to 010, 024)

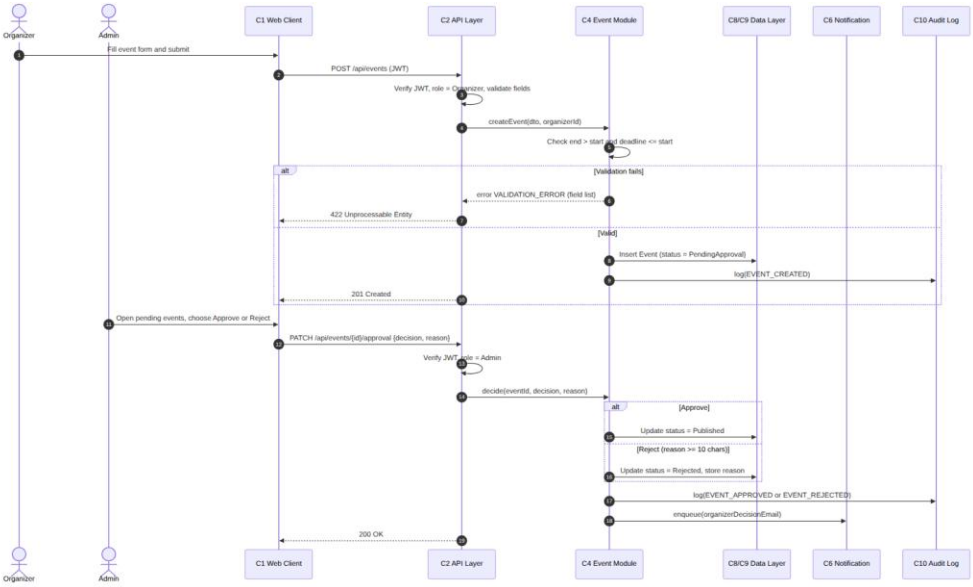


Figure 4: Sequence diagram SD-2 - Organizer creates event, Admin approves

SD-3: Cancel registration and promote from waitlist (EMS-F-018, 019)

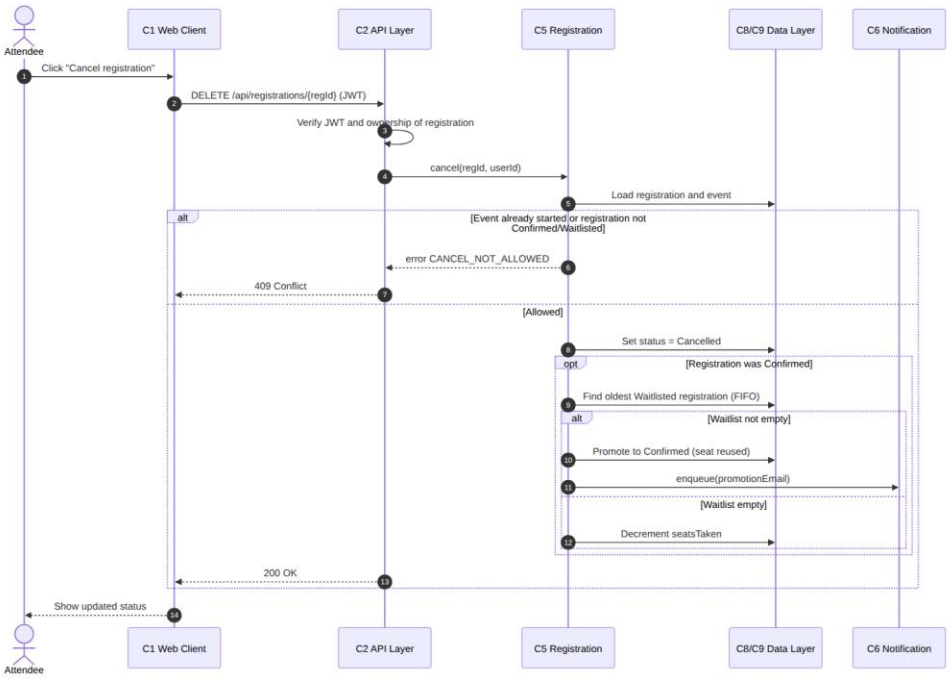


Figure 5: Sequence diagram SD-3 - cancellation with FIFO waitlist promotion

4.3 API Design

Conventions: base path /api; JSON bodies; authentication through the session cookie set at login; all endpoints require HTTPS; dates in ISO 8601 UTC; list endpoints accept page and limit (default 10); errors use the format in section 4.4.

4.3.1 Auth & User API (component C3)

Endpoint	Role	Request	Success	Errors
POST /auth/register	Public	{name, email, password}	201 {id, name, role}	422 validation, 409 email exists
POST /auth/login	Public	{email, password}	200 {user}; sets cookie	401 invalid credentials, 423 locked, 429 rate limit
POST /auth/logout	Any user	-	204	401
POST /auth/forgot-password	Public	{email}	202 (same answer whether or not email exists)	429
POST /auth/reset-password	Public	{token, newPassword}	200	400 token invalid or expired, 422
PUT /users/me	Any user	{name?, phone?}	200 {user}	401, 422
PATCH /admin/users/{id}	Admin	{active?, role?}	200 {user}	401, 403, 404, 422

Detailed specification: POST /api/auth/login

```
Request:  { "email": "asha@pes.edu", "password": "Event123" }
```

```
Response 200: { "user": { "id": "66f1...", "name": "Asha", "role": "Attendee" } }

   Set-Cookie: ems_token=<jwt>; HttpOnly; Secure; SameSite=Strict; Max-Age=3600

Response 401: { "error": { "code": "INVALID_CREDENTIALS", "message": "Email or
password is incorrect" } }

Response 423: { "error": { "code": "ACCOUNT_LOCKED", "message": "Try again in 12
minutes", "retryAfterSeconds": 720 } }
```

4.3.2 Event API (component C4)

Endpoint	Role	Request	Success	Errors
GET /events	Public	query: q, category, from, to, available, page, limit	200 {items[], page, total}	422 bad query
GET /events/{id}	Public (Published)	-	200 {event, remainingSeats}	404
POST /events	Organizer	{title, description, categoryId, venue, startAt, endAt, registrationDeadline, capacity}	201 {event} (status Pending Approval)	401, 403, 422 with field list
PATCH /events/{id}	Organizer (owner)	same fields (partial)	200 {event}	403 not owner, 404, 409 capacity below confirmed, 422
PATCH /events/{id}/approval	Admin	{decision: approve|reject, reason?}	200 {event}	403, 404, 409 not pending, 422 reason under 10 chars
POST /events/{id}/cancel	Organizer (owner)	-	200 {event}	403, 404, 409 already started or cancelled
GET/POST/PATCH /admin/categories	Admin	{name, active?}	200/201 {category}	403, 409 duplicate name

4.3.3 Registration API (component C5)

Endpoint	Role	Request	Success	Errors
POST /events/{id}/registrations	Attendee	-	201 {registrationId, regCode, status: Confirmed|Waitlisted, waitlistPosition?}	401, 403, 404, 409 DUPLICATE_REGISTRATION or EVENT_CLOSED
GET /registrations/me	Attendee	query: status?	200 {items[]}	401
DELETE /registrations/{id}	Attendee (owner)	-	200 {status: Cancelled}	403, 404, 409 CANCEL_NOT_ALLOWED

Detailed specification: POST /api/events/{id}/registrations

```
Request:  (no body; event id in path; attendee taken from the token)

Response 201: { "registrationId": "66f2...", "regCode": "REG-7F3K9Q", "status":
"Confirmed" }
```

```
Response 201: { "registrationId": "66f3...", "regCode": "REG-2M8XPA", "status":
"Waitlisted", "waitlistPosition": 3 }

Response 409: { "error": { "code": "DUPLICATE_REGISTRATION", "message": "Already
registered" } }

Response 409: { "error": { "code": "EVENT_CLOSED", "message": "Registration is closed
for this event" } }

Rules: Use one MongoDB transaction for seat allocation and booking creation. Roll back both
if either fails. A Checked-in booking still occupies its seat.
```

4.3.4 Attendance & Reporting API (component C7)

Endpoint	Role	Request	Success	Errors
POST /events/{id}/checkin	Organizer (owner)	{regCode}	200 {registration: Checked-in, checkedInAt}	403, 404, 409 NOT_CONFIRMED / ALREADY_CHECKED_IN, 422 OUTSIDE_WINDOW
GET /events/{id}/report	Organizer (owner)	-	200 {event, summary, registrations} summary includes attendanceRate	403, 404
GET /events/{id}/report.csv	Organizer (owner)	-	200 text/csv	403, 404
GET /admin/summary	Admin	-	200 {eventsByStatus, usersByRole, registrationsPerDay[]}	403

4.4 Error Handling, Logging & Monitoring

Error handling

All errors pass through one central error-handling middleware that returns a uniform body and never exposes stack traces or internal details to clients.

```
{ "error": { "code": "VALIDATION_ERROR", "message": "Invalid event data",

    "fields": [ { "field": "endAt", "message": "End must be after start" } ],

    "requestId": "b7c1e0" } }
```

HTTP	Error code	When
400	BAD_REQUEST / TOKEN_INVALID	Malformed request or invalid reset token
401	UNAUTHENTICATED / INVALID_CREDENTIALS	Missing, expired or tampered token; wrong credentials
403	FORBIDDEN	Role or ownership check failed
404	NOT_FOUND	Resource does not exist or is not visible to the user
409	DUPLICATE_REGISTRATION, EVENT_CLOSED, CANCEL_NOT_ALLOWED, NOT_CONFIRMED, ALREADY_CHECKED_IN	Business-rule conflicts

HTTP	Error code	When
422	VALIDATION_ERROR, OUTSIDE_WINDOW	Field validation failed (field list returned)
423	ACCOUNT_LOCKED	Too many failed logins
429	RATE_LIMITED	Rate limit exceeded
500	INTERNAL_ERROR	Unexpected failure; generic message with requestId

The client shows field errors where available. Profile updates return a general validation message. SMTP delivery is asynchronous, with retries and delivery logs.

Logging

- Structured JSON application logs (level, time, requestId, route, status, duration). Passwords, tokens, reset links and full email bodies are never logged (EMS-SEC-001).
- Audit log collection (EMS-SEC-006): user ID, action, IP, timestamp; retained 90 days.

Monitoring

• GET /health reports application and database status. Local samples cover the test run, while the 30-day availability target remains unverified.

- Tracked indicators: request latency (95th percentile), 5xx error rate, email failure rate, login failure rate, number of locked accounts.

4.5 UX Design

Screen	Design notes
Home / Event list	Card grid; search bar and filters at the top; 'Seats left' badge; pagination; 5 clicks or fewer to register (EMS-NF-004).
Event detail	Clear primary button: Register, Join waitlist, or Registered (disabled) depending on state.
My Registrations	Status chips (Confirmed, Waitlisted, Cancelled, Checked-in) and QR code per registration.
Organizer area	Create-event form with inline validation; event dashboard with counters and check-in box.
Admin console	Tabs: Pending events, Users, Categories, Summary.
General	Responsive layout (360 to 1920 px), sufficient colour contrast and labelled form fields for Lighthouse accessibility 90+ (EMS-NF-009); confirmation dialogs for destructive actions (EMS-SAF-001); all times shown in IST.

4.6 Open Issues & Next Steps

• The demo runs on localhost. Deployed TLS, redirects and the ZAP baseline remain unexecuted.

• Check-in supports manual code entry, camera scans and uploaded QR images. Physical-device permissions remain a manual check.

• See docs/DEVIATIONS.md for the implementation changes and validation limits.

5. Appendices

5.1 Glossary

EMS, SRS, STP, SAD, RBAC, JWT, REST, STRIDE, ADR, DAL, TLS, SMTP, IST, UTC, QR. See SRS section 1.4 and Appendix A.

5.2 References

- IEEE 42010 (architecture description) and IEEE 1016 (software design descriptions)
- OWASP Top 10 (2021) and ASVS 4.0
- NIST SP 800-160 (systems security engineering)
- EMS SRS v1.0 and STP v1.0

5.3 Tools

Mermaid, Graphviz and draw.io for diagrams; Postman or Swagger (OpenAPI) for API documentation; GitHub for source control, issues and project board; JMeter, Jest, Supertest and Lighthouse for testing.

5.4 Repository layout

```
ems/

  README.md

  docs/            SRS, SAD, STP (this document set) and diagram sources

  frontend/        React app (src/pages, src/components, src/api)

  backend/

    src/app.js

    src/middleware/   auth.js, rbac.js, ownership.js, validate.js, rateLimit.js,
errorHandler.js

    src/models/       user, event, category, registration, notificationLog, auditLog

    src/modules/      auth/, events/, registrations/, notifications/, attendance/,
reports/, admin/, audit/

    tests/           unit and API tests (Jest, Supertest)

docs/testing/ reports, test cases and localhost test helpers

  .github/         workflows/ci.yml, pull_request_template.md, ISSUE_TEMPLATE/
```